

Práctica N.º 1

Widgets básicos y manejo del estado con setState

Asignatura	Desarrollo de Aplicaciones Móviles (INF 662)	•	6.º semestre
Docente	M. Sc. Lic. Huáscar Fedor Gonzales Guzmán		
Base	Flutter (canal estable) • Dart 3 • solo SDK, sin paquetes externos		
Modalidad	Individual (trabajo independiente)	•	Fecha de entrega: 01 / 09 / 2026

En esta práctica consolidarás lo trabajado en los Capítulos 1 y 2 de la asignatura: la construcción de interfaces con widgets básicos y la interactividad mediante el patrón setState. Desarrollarás de forma individual las cinco aplicaciones que se describen a continuación.

01 Objetivos

- Construir interfaces con los widgets fundamentales de Flutter (Scaffold, AppBar, Column, Row, Container, Text, botones y campos de entrada).
- Aplicar correctamente setState para reflejar cambios de estado en la interfaz.
- Capturar y validar entradas del usuario (texto y valores numéricos) de forma segura.
- Gestionar colecciones en memoria (agregar, actualizar y eliminar elementos) sin persistencia.
- Producir aplicaciones ordenadas, legibles y con un manejo razonable de casos límite.

02 Contenidos que se evalúan

La práctica se limita a los siguientes contenidos. No utilices Provider, sqflite, ni llamadas asíncronas a servicios externos: se abordarán en prácticas posteriores.

ALCANCE

Widgets básicos: Scaffold, AppBar, Column, Row, Wrap, Container, Card, Text, Icon, ElevatedButton, IconButton, TextField, Slider, Checkbox, ListView.builder, AlertDialog, SnackBar.

Estado: StatefulWidget + setState, TextEditingController (con dispose), conversión segura con double.tryParse / int.tryParse.

03 Instrucciones generales

- Trabajo individual. Cada estudiante entrega su propia solución; el código copiado invalida la calificación de las partes involucradas.
- Un proyecto Flutter por aplicación (o un repositorio con una carpeta por app, nombradas app1_moodtracker, app2_tipsplit, ...).
- Cada aplicación usa al menos un StatefulWidget y actualiza su interfaz exclusivamente con setState.

- Solo se permite el SDK de Flutter; no agregues paquetes externos salvo que el enunciado lo pida.
- Formatea el código con dart format y usa nombres de variables y widgets descriptivos.
- Incluye por cada app una captura de pantalla y un breve README (qué hace y cómo ejecutarla).

04 Aplicaciones a desarrollar

Desarrolla las cinco aplicaciones. Cada una vale 20 puntos (ver criterios de evaluación). Los retos opcionales otorgan puntaje adicional (bonus) hasta un máximo de 10 puntos sobre el total.

APP 1 MoodTracker — ¿Cómo te sientes hoy?

Objetivo. Introducir la actualización de estado con `setState` y la composición de layout con `Row` / `Column` / `Wrap`.

Aplicación de una sola pantalla que muestra un conjunto de estados de ánimo (feliz, neutral, triste, enojado, cansado) representados con íconos o emoji. Al tocar un ánimo, su contador aumenta en uno. La pantalla muestra el total de registros del día y el ánimo predominante. Un botón «Reiniciar día» pone todos los contadores en cero.

REQUISITOS MÍNIMOS

- Al menos cuatro estados de ánimo, cada uno con su contador visible.
- El total general se actualiza en tiempo real al tocar cualquier ánimo.
- Botón «Reiniciar día» que devuelve todos los contadores a cero.
- Toda actualización de la interfaz debe realizarse mediante `setState`.

Widgets clave
ELEVATEDBUTTON

SCAFFOLD

APPBAR

COLUMN

ROW / WRAP

ICONBUTTON

CARD

RETO OPCIONAL (BONUS)

Resalta visualmente (color o tamaño) la tarjeta del ánimo con más registros.

APP 2 TipSplit — Calculadora de propina

Objetivo. Capturar entrada numérica y recalcular en vivo combinando `TextField`, `Slider` y `setState`.

El usuario ingresa el monto de la cuenta, elige el porcentaje de propina con un `Slider` (0–30 %) y ajusta el número de personas con botones + / –. La app muestra la propina, el total y el monto por persona, actualizándose en cada cambio.

REQUISITOS MÍNIMOS

- `TextField` numérico con `TextEditingController` y conversión segura (`double.tryParse`).
- `Slider` para el porcentaje de propina, mostrando el valor actual.
- Controles + / – para el número de personas (mínimo 1).
- Resultados (propina, total y monto por persona) presentados en una `Card`, con dos decimales.

Widgets clave

TEXTFIELD

SLIDER

ICONBUTTON

CARD

TEXT

SETSTATE

ADVERTENCIA

La app debe tolerar entradas vacías o inválidas sin cerrarse ni mostrar NaN.

RETO OPCIONAL (BONUS)

Agrega chips de propina rápida (10 %, 15 %, 20 %) que ajusten el Slider al tocarlos.

APP 3 ColorLab — Mezclador de color RGB

Objetivo. Vincular varios controles continuos a una única salida visual mediante `setState`.

Tres sliders (R, G y B, de 0 a 255) controlan un panel de vista previa que cambia de color en vivo. Se muestran los valores RGB y el código hexadecimal. El texto sobre la vista previa cambia entre blanco y negro según el brillo, para mantener el contraste.

REQUISITOS MÍNIMOS

- Tres sliders independientes para los canales R, G y B.
- Panel de vista previa (Container con BoxDecoration) que refleja el color en tiempo real.
- Mostrar el valor RGB y el código hexadecimal correspondiente.
- Botón «Color aleatorio» que genera los tres canales al azar.

Widgets clave
(DART:MATH)

SLIDER
SETSTATE

CONTAINER / BOXDECORATION

TEXT

ELEVATEDBUTTON

RANDOM

RETO OPCIONAL (BONUS)

Muestra el código HEX en mayúsculas y permite «copiarlo» confirmando con un SnackBar.

APP 4 QuickList — Lista de pendientes rápida

Objetivo. Gestionar una colección en memoria (alta, marcado y baja) con `setState`, sin navegación ni persistencia.

Pantalla única donde el usuario agrega tareas con un TextField, las marca como completadas con un Checkbox y las elimina con un ícono. Un contador muestra las tareas pendientes y las completadas. Cuando no hay tareas, se muestra un estado vacío.

REQUISITOS MÍNIMOS

- Agregar tareas desde un TextField; no se permite guardar texto vacío.
- Lista construida con `ListView.builder`.
- Checkbox por tarea para marcarla o desmarcarla (texto tachado al completarse).
- Posibilidad de eliminar cada tarea individualmente.
- Estado vacío visible y contador de pendientes / completadas.

Widgets clave

TEXTFIELD

LISTVIEW.BUILDER

CHECKBOX

ICONBUTTON

TEXT

SETSTATE

RETO OPCIONAL (BONUS)

Añade un filtro (Todas / Pendientes / Completadas) usando ChoiceChip.

APP 5

GuessIt — Adivina el número

Objetivo. Integrar lógica de juego, entrada de datos y actualización condicional de la interfaz con `setState`.

La app elige un número secreto entre 1 y 100. El usuario ingresa intentos y recibe pistas («más alto» / «más bajo») hasta acertar. Se cuenta el número de intentos y se puede reiniciar la partida con un número nuevo.

REQUISITOS MÍNIMOS

- Número secreto aleatorio (Random) generado al iniciar y al reiniciar.
- Entrada del intento validada dentro del rango 1–100.
- Retroalimentación clara (mayor / menor / acertó) y contador de intentos.
- Mensaje de victoria destacado (Text o AlertDialog) y botón «Jugar de nuevo».

Widgets clave

TEXTFIELD

ELEVATEDBUTTON

ALERTDIALOG

TEXT

RANDOM

SETSTATE

RETO OPCIONAL (BONUS)

Guarda y muestra el mejor puntaje (menor número de intentos) de la sesión.

05 Producto entregable

- Código fuente de las cinco aplicaciones (proyectos Flutter o carpetas dentro de un repositorio).
- Una captura de pantalla por aplicación mostrando su funcionamiento.
- Un README por aplicación con una descripción breve y las instrucciones de ejecución.
- Opcional: un video corto (máx. 3 minutos) demostrando las cinco apps.

06 Criterios de evaluación

Cada aplicación se evalúa sobre 20 puntos con la siguiente distribución. La calificación final es el promedio de las cinco aplicaciones.

Criterio	Descripción	Puntos
Funcionalidad	Cumple todos los requisitos mínimos del enunciado.	8
Estado y widgets	Uso correcto de <code>setState</code> y de los widgets básicos indicados.	5
Interfaz	Diseño claro y ordenado: layout, espaciado y legibilidad.	3

Criterio	Descripción	Puntos
Casos límite	Manejo de entradas vacías o inválidas y del estado vacío.	2
Código	Organización y legibilidad: nombres, estructura, dart format.	2
Total por aplicación		20

NOTA

Los retos opcionales se acreditan como bonus solo si los requisitos mínimos de la aplicación están completos. Una app que se cierra ante una entrada inválida no obtiene el puntaje de «Casos límite».

07 Forma y fecha de entrega

- Sube tu solución a un repositorio de GitHub personal (público o con acceso al docente) y envía el enlace en un archivo de texto a la plataforma virtual de la universidad.
- Verifica que el proyecto compile con flutter run antes de entregar.

08 Recursos de apoyo

EJEMPLO

Catálogo de widgets: docs.flutter.dev/ui/widgets • Referencia de API: api.flutter.dev

Repasa las Guías de Laboratorio N.º 3, 5, 6, 7 y 8, que cubren layout con Scaffold/Column/Row, formularios y setState, el conversor de unidades, el registro de estudiantes y el cronómetro.